ELI — Lora Pipeline

ELI v2.2.7

August 2026

Contents

LoRA fine-tuning — audit & pipeline (2026-06-07)

What was found (audit)

The LoRA mechanisms in `eli/learning/` were **individually correct but orphaned and phi-hardcoded**:

- **Algorithm — correct.** `lora_trainer.run_training` is a proper PEFT LoRA loop: tokenize (truncation/pad, `labels=input_ids`) → `AutoModelForCausalLM` (eager attn, fp16 on CUDA, gradient checkpointing) → `LoraConfig` → `get_peft_model` → `Trainer` with `DataCollatorForLanguageModeling(mlm=False)` → `trainer.train()` → `save_pretrained` (adapter + tokenizer). Safety contract is sound (dry-run default, `--execute` required, reviewed rows only, **GGUF never trained directly**, adapter never overwritten).
- **Stages exist** but as separate CLI scripts with no orchestrator: guard plan (`lora_trainer_guard`) → `training_preflight` → `dataset_builder` / `export_trainable_dataset` / `merge_reviewed_datasets` → `lora_trainer` → `lora_eval`. Nothing chained them.
- **Orphaned** — nothing outside `eli/learning/` called any of them (no executor action, router, GUI button, scheduled task, or self-upgrade hook). ELI could not trigger or even report on LoRA.
- **Not model-agnostic** — `target_modules` defaulted to phi-3's `["qkv_proj"]`.

What was wired/fixed

- **Model-agnostic target modules** (`lora_trainer._resolve_target_modules`): derives LoRA targets from the LOADED architecture (scans for known projection Linear leaves: q/k/v/o_proj, qkv_proj, gate/up/down_proj, c_attn, query_key_value, …), honours an explicit adapter-config override, and falls back to PEFT's architecture-agnostic `"all-linear"`. No hardcoded architecture on the train path.
- **Pipeline DAG** (`eli/learning/lora_pipeline.py::run_pipeline`): the explicit ordered chain **preflight → build_job → [train] → eval(inspect)**, reusing the existing modules. **Dry-run by default** (runs every gate, touches no GPU, writes no adapter — safe from chat); real training only `execute=True`.
- **Wired into ELI:**
  - `LORA_STATUS` action (read-only): preflight readiness per target.
  - `LORA_TRAIN` action: runs the pipeline DAG — **dry-run from chat**; real training only via the scheduled task / explicit GUI (execute flag not exposed in chat).

  - Router: "lora status" / "is lora ready" → LORA_STATUS; "train a lora" / "fine-tune yourself" / "run lora training" → LORA_TRAIN.
  - Scheduled `lora` kind (`_worker_lora`): "train a lora overnight [N steps]" → `SCHEDULE_TASK` runs `run_pipeline(execute=True)` unattended.
  - Manifest: 216 capabilities (includes LORA_STATUS + LORA_TRAIN).
- Tests: `tests/test_lora_pipeline.py` (target-module resolver, DAG order + dry-run no-train, both actions, routing, scheduled kind).

Still phi-profiled (recorded, your call)

The training **target profiles** are still `eli_phi` / `eli_phi_ultra` (and `bootstrap_phi3_base.py`, the rope shim) — the chosen training base is Phi-3. The *algorithm* is now model-agnostic; making the **base profile** swappable (any HF causal-LM dir, not just Phi-3) is a larger, separate change — deferred to your go-ahead. LoRA needs a Hugging Face model directory, not the GGUF ELI runs for inference, so training a new brain → adapter → merge → convert-to-GGUF is the intended flow.